

## Practical Week 12

**Task 1** - Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        /* Wait for an empty space */
        sem_wait(&empty);

        /* Enter critical section */
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        /* Increase number of filled spaces */
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    long long sum = 0;

    for (int i = 0; i < ITEMS; i++)
    {
        /* Wait for an available item */
        sem_wait(&full);

        /* Enter critical section */
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        /* Increase number of empty spaces */
    }
}
```

```

        sem_post(&empty);

        sum += item;
    }

    printf("Consumer processed %d items. Sum = %lld\n",
        ITEMS, sum);

    return NULL;
}

void run_test(int size)
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    buffer_size = size;

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL)
    {
        perror("Memory allocation failed");
        exit(1);
    }

    in = 0;
    out = 0;

    /* Initialize semaphores */
    sem_init(&empty, 0, buffer_size);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    clock_t start = clock();

    /* Create producer and consumer */
    pthread_create(&producer_thread,
        NULL,
        producer,
        NULL);

    pthread_create(&consumer_thread,
        NULL,
        consumer,
        NULL);

    /* Wait for both threads */
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_t end = clock();

    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Buffer Size: %d\n", buffer_size);
    printf("Execution Time: %.6f seconds\n", time_taken);
}

```

```

    /* Destroy synchronization objects */
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    free(buffer);
}

int main()
{
    printf("=====\n");
    printf("    PRODUCER-CONSUMER PROBLEM\n");
    printf("=====\n");

    printf("\nTesting different buffer sizes...\n\n");

    run_test(5);
    run_test(10);
    run_test(50);
    run_test(100);

    printf("\nAll tests completed successfully.\n");

    return 0;
}

```

```

mohana_priya@AMMU:~$ cd task1_threads
-bash: cd: task1_threads: No such file or directory
mohana_priya@AMMU:~$ ls
fork_process      memory_analysis  myShell.c        process_states
fork_process.c    memory_analysis.c practical6         process_states.c
linux_io_project  myShell          practical_memory  task30
mohana_priya@AMMU:~$ nano mutex_counter.c
mohana_priya@AMMU:~$ gcc mutex_counter.c -o mutex_counter -pthread
mohana_priya@AMMU:~$ ls
fork_process      memory_analysis  mutex_counter.c  practical6         process_states.c
fork_process.c    memory_analysis.c myShell          practical_memory  task30
linux_io_project  mutex_counter    myShell.c        process_states
mohana_priya@AMMU:~$ ./mutex_counter
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000
mohana_priya@AMMU:~$ for i in {1..5}; do ./mutex_counter; done
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000
mohana_priya@AMMU:~$ ./race_condition
-bash: ./race_condition: No such file or directory
mohana_priya@AMMU:~$ ./mutex_counter
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000
mohana_priya@AMMU:~$ pthread_mutex_lock(&lock);
-bash: syntax error near unexpected token '&'
mohana_priya@AMMU:~$ time ./race_condition
-bash: ./race_condition: No such file or directory

real    0m0.002s
user    0m0.001s
sys      0m0.001s
mohana_priya@AMMU:~$ time ./mutex_counter
Initial counter: 0
Final counter: 4000000
Expected counter: 4000000

real    0m0.207s
user    0m0.269s
sys      0m0.367s
mohana_priya@AMMU:~$ mkdir producer_consumer
mohana_priya@AMMU:~$ cd producer_consumer
mohana_priya@AMMU:~/producer_consumer$ pwd
/home/mohana_priya/producer_consumer
mohana_priya@AMMU:~/producer_consumer$ nano producer_consumer.c
mohana_priya@AMMU:~/producer_consumer$ gcc producer_consumer.c -o producer_consumer -pthread
mohana_priya@AMMU:~/producer_consumer$ ls
producer_consumer  producer_consumer.c
mohana_priya@AMMU:~/producer_consumer$ ./producer_consumer

```

```

mohana_priya@AMMU:~/producer_consumer$ ./producer_consumer

```

# PRODUCER-CONSUMER PROBLEM

Testing different buffer sizes...

```

Consumer processed 100000 items. Sum = 4999950000
Buffer Size: 5
Execution Time: 0.854672 seconds

```

```

Consumer processed 100000 items. Sum = 4999950000
Buffer Size: 10
Execution Time: 0.434974 seconds

```

```

Consumer processed 100000 items. Sum = 4999950000
Buffer Size: 50
Execution Time: 0.076118 seconds

```

```

Consumer processed 100000 items. Sum = 4999950000
Buffer Size: 100
Execution Time: 0.055665 seconds

```

All tests completed successfully.

```

mohana_priya@AMMU:~/producer_consumer$ |

```

Task 2 - Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed.\n");

    return 0;
}
```

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: Released both resources\n");

    return NULL;
}

void *thread2_function(void *arg)
{
    /*
     * Resource ordering:
     * Always acquire Resource 1 first,
     * then Resource 2.
     */

    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: Released both resources\n");

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed without deadlock.\n");

    return 0;
}

```

```

mohana_priya@AMMU:~$ ^C
mohana_priya@AMMU:~$ mkdir task2_deadlock
cd task2_deadlock
mohana_priya@AMMU:~/task2_deadlock$ nano deadlock.c
mohana_priya@AMMU:~/task2_deadlock$ gcc deadlock.c -o deadlock -pthread
mohana_priya@AMMU:~/task2_deadlock$ gcc deadlock.c -o deadlock -pthread
mohana_priya@AMMU:~/task2_deadlock$ ./deadlock
Thread 1: Locking Resource 1
Thread 2: Locking Resource 2
Thread 1: Waiting for Resource 2
Thread 2: Waiting for Resource 1
^C
mohana_priya@AMMU:~/task2_deadlock$ nano deadlock_prevention.c
mohana_priya@AMMU:~/task2_deadlock$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
mohana_priya@AMMU:~/task2_deadlock$ ./deadlock_prevention
Thread 1: Locking Resource 1
Thread 2: Locking Resource 1
Thread 1: Locking Resource 2
Thread 1: Acquired both resources
Thread 1: Released both resources
Thread 2: Locking Resource 2
Thread 2: Acquired both resources
Thread 2: Released both resources
Program completed without deadlock.
mohana_priya@AMMU:~/task2_deadlock$ ./deadlock
Thread 1: Locking Resource 1
Thread 2: Locking Resource 2
Thread 1: Waiting for Resource 2
Thread 2: Waiting for Resource 1
^C
mohana_priya@AMMU:~/task2_deadlock$ ./deadlock_prevention
Thread 1: Locking Resource 1
Thread 2: Locking Resource 1
Thread 1: Locking Resource 2
Thread 1: Acquired both resources
Thread 1: Released both resources
Thread 2: Locking Resource 2
Thread 2: Acquired both resources
Thread 2: Released both resources
Program completed without deadlock.
mohana_priya@AMMU:~/task2_deadlock$ |

```